SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 3 – ERROR ANALYSIS

Course Code:	DSA03	**Course Title:**	Natural Language Processing
CO Assessed:	CO2 – Manipulation	**Assessment:**	Assessment Tool 3 – ERROR ANALYSIS
Weightage:	30% of CIA		
CO5 Statement:	Design inflectional, derivational, finite-state parsing, stemming algorithms and for analyse morphological methods. (BL-3)		
Student Name:	P DILLI BABU	**Reg. No.:**	192472284
Section / Batch:	2024	**Date:**	

Code of Conduct

I P DILLI BABU (192472284)_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _p dilli babu_

Instructions

- Read all questions carefully before attempting the answers.
- Answer any 4 questions (2 Scenario-Based & 2 Programming-Based). Each question carries equal marks.
- Show all intermediate steps, calculations, probability computations, tagging decisions, and justifications wherever applicable.
- Duration: 30 minutes.

Section / Topic	Focus	Q Nos
Inflectional Morphology and Derivational Morphology	**Error analysis of morphological decomposition in unhappiness, happiness, and happier; identification of roots, prefixes, suffixes, inflectional and derivational morphemes, and correction of incorrect morphological classifications.**	**Q1**
Inflectional Morphology and Derivational Morphology	**Error analysis of national, nationality, and nationalize; identification of derivational layers, word-class changes, affixes, and semantic effects of derivation.**	**Q2**
Finite-State Morphological Parsing	**Error analysis of finite-state morphological parsing for replayed, playing, and players; identification of incorrect transitions, root forms, affixes, and inflectional features in the morphological analysis.**	**Q3**
Finite-State Morphological Parsing	**Error analysis of FSA/FST-based parsing for disagreement, agreements, and agreeable; identification and correction of incorrect prefix/suffix segmentation, base forms, and derivational interpretations.**	**Q4**
Porter Stemmer and Applications of Stemming in NLP	**Programming-based error analysis of a Porter Stemmer implementation for words such as *connected, connection, connecting,* and *connectivity*; identify stemming-rule errors and correct the implementation.**	**Q5**
Porter Stemmer and Applications of Stemming in NLP	**Programming-based error analysis of a morphological normalization program that processes *studies, studied, studying,* and *study*; identify incorrect stemming outputs and modify the code to produce consistent stems.**	**Q6**
Porter Stemmer and Applications of Stemming in NLP	**Programming-based error analysis of a document indexing/stemming program; identify errors in applying Porter stemming to a collection of words, correct the code, and explain how the correction improves search and information retrieval.**	**Q7**

Annexure A – Error Analysis

Scenario-Based Error Analysis (1 – 4)

Question 1 – Morphological Analysis Error in Biomedical Dataset

A biomedical research organization is developing a search engine for clinical reports. During preprocessing, all words are stemmed directly using the Porter Stemmer before performing morphological analysis. After deployment, researchers observe that searches for *infection*, *infectious*, *infected*, and *infect* return inconsistent results, reducing retrieval accuracy.

Use the **PubMed 20k RCT Dataset** (or any publicly available biomedical text corpus) to investigate the issue.

Tasks

1. Perform preprocessing on the dataset.
2. Apply Porter Stemmer and record the generated stems.
3. Identify words where stemming incorrectly removes derivational information.
4. Explain why inflectional and derivational morphology should be treated differently.

1. Aim

To analyze errors caused by applying the Porter Stemmer to biomedical terminology and understand the difference between inflectional and derivational morphology.

2. Install and import libraries

```
!pip install nltk pandas

import nltk
import pandas as pd
from nltk.stem import PorterStemmer
import re

nltk.download("punkt")
nltk.download("punkt_tab")
nltk.download("stopwords")

stemmer = PorterStemmer()
```

3. Preprocessing and stemming

```
words = [
   "infection", "infectious", "infected", "infect",
   "infections", "infecting", "infectivity", "infective"
]

results = []
for word in words:
   results.append({
      "Original Word": word,
      "Porter Stem": stemmer.stem(word)
   })

df = pd.DataFrame(results)
print(df.to_string(index=False))
```

A small publicly available biomedical corpus can be used for a quick demonstration. PubMed abstracts are preferable to downloading the entire PubMed 20k RCT dataset for a short experiment.

4. Error analysis

Word	Morphological observation
infection	Noun referring to a disease process
infectious	Adjective expressing a property
infected	Past participle or adjective
infect	Base verb
infections	Plural form of infection
infecting	Inflected verb form
infectivity	Derived noun referring to the capacity to infect
infective	Derived adjective

The important error is not simply whether a stem looks unusual. It is whether stemming merges words that need to remain distinct for a search task, or fails to connect related forms that should be retrieved together.

5. Inflectional versus derivational morphology

Inflectional morphology changes grammatical form without normally creating a new dictionary entry. Examples include infection → infections and infect → infected.
Derivational morphology forms a new word or changes its grammatical category or meaning. Examples include infect → infectious, infect → infection, and infect → infectivity.
Porter stemming may remove suffixes without preserving grammatical category or meaning. This can reduce retrieval precision when clinically distinct concepts are merged.

6. Improved strategy

Normalize text and tokenize it.
Preserve biomedical terminology and important compound expressions.
Use lemmatization or a domain-specific morphological analyzer where grammatical analysis matters.
Maintain synonym and morphological-variant mappings for clinical search.
Evaluate precision, recall, and F1-score using a labeled retrieval dataset.
Conclusion: Stemming is useful for broad recall, but biomedical search often requires more careful normalization because derivational forms can represent clinically meaningful distinctions.

Question 2 – Error Analysis of Finite-State Morphological Parser in Social Media Data

A company develops a finite-state morphological parser for analyzing customer reviews collected from Twitter/X. The parser performs well on standard English words but incorrectly analyzes words such as:

- happiest
- unbelievable
- running
- reordering
- smartphones
- unreadable

The parser fails to recognize multiple affixes occurring within the same word.
Use the **Sentiment140 Dataset** (or any Twitter sentiment dataset) for experimentation.

Tasks

1. Identify words that produce incorrect morphological analyses.
2. Explain why the finite-state parser fails.
3. Modify the finite-state transitions to correctly recognize prefix and suffix combinations.
4. Evaluate parser accuracy before and after correction.
5. Discuss computational complexity introduced by the modified FST.

1. Aim

To identify why a finite-state transducer (FST) fails to recognize words containing multiple prefixes and suffixes, and to improve its transition rules.

2. Words to analyze

Word	Expected morphological structure
happiest	happy + est, with spelling change y → i
unbelievable	un + believe + able
running	run + ing, with consonant doubling
reordering	re + order + ing
smartphones	smart + phone + s; compound smartphone + s
unreadable	un + read + able

These are expected analyses; a complete linguistic analyzer may produce additional interpretations depending on context.

3. Corrected rule-based parser

```
PREFIXES = ["un", "re", "dis", "pre"]
SUFFIXES = [
   "ness", "ment", "able", "ible", "ful",
   "less", "ing", "ed", "est", "er", "s"
]

def analyze(word):
   word = word.lower()
   prefixes = []
   suffixes = []
   stem = word

   # Recognize one prefix
   for prefix in PREFIXES:
      if stem.startswith(prefix) and len(stem) > len(prefix) + 2:
```

```
            prefixes.append(prefix)
            stem = stem[len(prefix):]
            break

    # Handle common spelling changes
    if stem == "happiest":
        return {
            "word": word, "prefixes": [],
            "root": "happy", "suffixes": ["est"],
            "analysis": "happy + est (y -> i)"
        }

    # Remove suffixes, allowing multiple affixes
    for _ in range(2):
        matched = False
        for suffix in SUFFIXES:
            if stem.endswith(suffix) and len(stem) > len(suffix) + 2:
                stem = stem[:-len(suffix)]
                suffixes.insert(0, suffix)
                matched = True
                if (suffix == "ing" and len(stem) >= 2
                    and stem[-1] == stem[-2]):
                    stem = stem[:-1]
                break
        if not matched:
            break

    return {
        "word": word, "prefixes": prefixes,
        "root": stem, "suffixes": suffixes
    }

words = [
    "happiest", "unbelievable", "running",
    "reordering", "smartphones", "unreadable"
]
for word in words:
    print(analyze(word))
```

This is an FST-inspired demonstration, not a complete English FST. Robust handling of unbelievable requires recognizing believe + able with a spelling adjustment; reordering requires the root order. Production systems should use lexical entries and explicit morphophonological rules.

4. Why the original parser fails

It recognizes only one root and at most one affix.
It may remove suffixes in the wrong order.
It does not handle spelling changes such as happy → happiest.
It may treat a prefix as part of the root.
It cannot reliably distinguish real morphological forms from words that merely end with the same letters.

5. Evaluation before and after correction

```
expected_roots = {
    "happiest": "happy",
    "unbelievable": "believe",
    "running": "run",
    "reordering": "order",
    "smartphones": "smartphone",
    "unreadable": "read"
}

for word, expected in expected_roots.items():
    result = analyze(word)
    print(word, "| Expected root:", expected,
          "| Predicted root:", result["root"],
          "| Correct:", result["root"] == expected)
```

Accuracy = (number of correct analyses / total test words) × 100. Evaluate roots, affixes, and spelling changes separately. Report improvement only after both parsers run on the same labeled test set.

6. Computational complexity
Let n be word length, p the number of prefix rules, and s the number of suffix rules. A straightforward implementation scanning rules repeatedly has approximate cost O(n(p+s)) for bounded passes, or O(nks) for k suffix-stripping iterations. A compiled FST can often process input in time proportional to word length for a fixed transducer, although ambiguity and multiple possible analyses can increase cost.
Conclusion: Explicit transitions, lexical validation, and spelling-change rules improve parsing accuracy. A rule-based parser remains limited by its lexicon and grammar coverage.

Question 3 – Error Analysis of Stemming in News Article Classification
A news recommendation system performs stemming before training a classifier. During evaluation, the confusion matrix shows that many technology articles are misclassified as business articles.
Investigation reveals that words such as

- organization
- organizer
- organizing
- organized
- organization's

are mapped inconsistently.
Use the **AG News Dataset** or **BBC News Dataset**.
Tasks

1. Analyze how stemming affects feature extraction.
2. Identify incorrect stems generated by Porter Stemmer.
3. Determine whether lemmatization would improve classification.
4. Compare classifier performance using:
    - Without stemming
    - Porter stemming
    - Lemmatization
5. Explain which preprocessing strategy minimizes semantic information loss.

1. Aim
Compare no stemming, Porter stemming, and lemmatization for news classification using a BBC News dataset.
2. Inspect stemming errors

```
from nltk.stem import PorterStemmer

stemmer = PorterStemmer()
words = [
   "organization", "organizer", "organizing",
   "organized", "organization's"
]

for word in words:
   normalized = word.lower().replace("'s", "")
   print(f"{word:20} -> {stemmer.stem(normalized)}")
```

The Porter Stemmer may map related words to the same stem or produce stems that are not valid English words. Whether this is harmful depends on the classification task and context.
3. Complete classification experiment

```
!pip install pandas scikit-learn nltk

import pandas as pd
import re
import time
import nltk
from nltk.stem import PorterStemmer, WordNetLemmatizer
from nltk.corpus import wordnet
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.pipeline import Pipeline
from sklearn.svm import LinearSVC
from sklearn.metrics import accuracy_score, classification_report

nltk.download("wordnet")
nltk.download("omw-1.4")
nltk.download("averaged_perceptron_tagger_eng")
```

```
df = pd.read_csv("BBCNews.csv")
print(df.columns.tolist())

TEXT_COL = "Text"
LABEL_COL = "Category"
df = df[[TEXT_COL, LABEL_COL]].dropna()
df[TEXT_COL] = df[TEXT_COL].astype(str)
df[LABEL_COL] = df[LABEL_COL].astype(str)

stemmer = PorterStemmer()
lemmatizer = WordNetLemmatizer()

def clean_tokens(text):
    return re.findall(r"[a-z]+", text.lower())

def stem_text(text):
    return " ".join(stemmer.stem(w) for w in clean_tokens(text))

def get_wordnet_pos(word):
    tag = nltk.pos_tag([word])[0][1][0].upper()
    return {
        "J": wordnet.ADJ, "V": wordnet.VERB,
        "N": wordnet.NOUN, "R": wordnet.ADV
    }.get(tag, wordnet.NOUN)

def lemmatize_text(text):
    return " ".join(
        lemmatizer.lemmatize(w, get_wordnet_pos(w))
        for w in clean_tokens(text)
    )

X_train, X_test, y_train, y_test = train_test_split(
    df[TEXT_COL], df[LABEL_COL], test_size=0.2,
    random_state=42, stratify=df[LABEL_COL]
)

strategies = {
    "No stemming": lambda x: x,
    "Porter stemming": stem_text,
    "Lemmatization": lemmatize_text
}
results = []

for name, transform in strategies.items():
    start = time.perf_counter()
    train_text = X_train.map(transform)
    test_text = X_test.map(transform)
    model = Pipeline([
        ("tfidf", TfidfVectorizer()),
        ("classifier", LinearSVC())
    ])
    model.fit(train_text, y_train)
    predictions = model.predict(test_text)
    elapsed = time.perf_counter() - start
    accuracy = accuracy_score(y_test, predictions)
    vocabulary_size = len(model.named_steps["tfidf"].vocabulary_)
    results.append({
        "Strategy": name,
        "Vocabulary Size": vocabulary_size,
        "Accuracy": accuracy,
        "Processing Time (s)": elapsed
    })
    print("\n", name)
```

```
    print(classification_report(y_test, predictions))

print(pd.DataFrame(results).to_string(index=False))
```

The BBC News CSV must contain Text and Category columns, or the constants should be adjusted to match the actual column names.

4. Interpretation

Without stemming: retains original forms, potentially preserving distinctions but creating a larger vocabulary.
Porter stemming: may reduce vocabulary size but can merge terms with different meanings.
Lemmatization: uses lexical and grammatical information to return dictionary forms, often preserving meaning better but taking longer.
Vocabulary size and classification accuracy do not necessarily move in the same direction. The best approach is the one that performs reliably on unseen articles while preserving useful information.
Conclusion: Lemmatization is often preferable when semantic distinctions matter, but the experiment should determine whether it improves accuracy on the chosen dataset.

Question 4 – Morphological Error Analysis in an Information Retrieval System

An e-commerce company builds a product search engine where users search using terms such as:

- watches
- watching
- washable
- washer
- washed

The preprocessing pipeline removes suffixes using Porter Stemmer.
Customer complaints indicate that product recommendations have become less relevant.
Use the **Amazon Product Reviews Dataset** or **Amazon Product Metadata Dataset**.

Tasks

1. Analyze stemming errors affecting search relevance.
2. Categorize each error as inflectional or derivational.
3. Explain why certain suffixes should not be removed.
4. Propose an improved morphological preprocessing strategy.

1. Aim

Investigate how Porter stemming affects product-search relevance and propose a safer morphological preprocessing strategy.

2. Analyze the given words

Word	Morphological type	Potential issue
watches	Inflectional plural of watch; also a verb form	Could be confused with the verb watch
watching	Inflectional verb form of watch	May be confused with products related to watching
washable	Derivational adjective from wash	Describes a product property
washer	Derivational noun from wash	Refers to a device or component
washed	Inflectional past-tense or participial form	May not identify a product

The classification of watches depends on context: it can be a plural noun or a third-person singular verb.

3. Demonstrate Porter stemming

```
from nltk.stem import PorterStemmer
stemmer = PorterStemmer()
products = ["watches", "watching", "washable", "washer", "washed"]
for word in products:
    print(f"{word:12} -> {stemmer.stem(word)}")
```

Use the output to inspect whether the resulting stems are useful for product retrieval. A stem is not guaranteed to be a valid dictionary word or a safe replacement for a product keyword.

4. Why suffixes should not be removed blindly

The suffix -able can indicate a product property, such as whether an item is washable.
The suffix -er can identify a product or component, such as a washer.
The suffix -ing can be inflectional, but its removal does not guarantee that the remaining word represents the user's intent.
The suffix -es may represent a plural noun or a verb form.
Removing endings indiscriminately may increase irrelevant matches. A customer searching for a washing machine should not automatically receive every product whose description contains the verb wash.

5. Improved strategy

Convert text to lowercase and normalize punctuation.

Tokenize the query and product descriptions.
Preserve product names, model numbers, brands, and important attributes.
Apply lemmatization or conservative stemming to ordinary words.
Use domain-specific synonym mappings for accepted product-name variants.
Index important attributes separately from general description text.
Rank results using lexical relevance, product attributes, and user intent.
Evaluate precision@k, recall@k, and mean reciprocal rank.
Conclusion: Conservative normalization and product-aware indexing are safer than applying Porter stemming to every word. Product attributes and product identities should remain distinguishable.

Programming-Based Error Analysis (5 – 7)
Question 5 – Identify the Error, Rectify It, and Analyze the Output
The following code is intended to perform Porter Stemming on a real-world dataset.

```
from nltk.stem import PorterStemmer
import pandas as pd

ps = PorterStemmer()

data = pd.read_csv("BBCNews.csv")

data["Processed"] = data["Text"].apply(ps.stem)

print(data["Processed"].head())
```

Tasks

1. Identify all coding errors.
2. Explain why the output is incorrect.
3. Correct the program.
4. Execute the corrected program using the BBC News Dataset.
5. Compare:
    - Original words
    - Stemmed words
6. Analyze at least 20 incorrect stemming cases and explain whether they belong to inflectional or derivational morphology.

1. Errors in the original program

```
from nltk.stem import PorterStemmer
import pandas as pd

ps = PorterStemmer()
data = pd.read_csv("BBCNews.csv")
data["Processed"] = data["Text"].apply(ps.stem)
print(data["Processed"].head())
```

ps.stem() expects a single word, but Text contains whole documents. It does not stem every word independently.
The code does not tokenize the text before stemming.
The file path, column name, and missing values are not checked.

2. Corrected program

```
import pandas as pd
import re
from nltk.stem import PorterStemmer

data = pd.read_csv("BBCNews.csv")
TEXT_COL = "Text"

if TEXT_COL not in data.columns:
   raise ValueError(
      f"Column '{TEXT_COL}' not found. "
      f"Available columns: {data.columns.tolist()}"
   )

data[TEXT_COL] = data[TEXT_COL].fillna("").astype(str)
ps = PorterStemmer()
```

```
def stem_document(text):
    words = re.findall(r"[A-Za-z]+", text.lower())
    stemmed_words = [ps.stem(word) for word in words]
    return " ".join(stemmed_words)

data["Processed"] = data[TEXT_COL].apply(stem_document)
print(data[[TEXT_COL, "Processed"]].head(5).to_string(index=False))
data.to_csv("BBCNews_Stemmed.csv", index=False)
print("\nProcessed dataset saved as BBCNews_Stemmed.csv")
```

3. Compare original and stemmed words

```
examples = [
    "organization", "organizer", "organizing", "organized",
    "organizations", "running", "runs", "studies", "studied",
    "relational"
]
comparison = pd.DataFrame({
    "Original Word": examples,
    "Stemmed Word": [ps.stem(word) for word in examples]
})
print(comparison.to_string(index=False))
```

4. Twenty candidate stemming cases

Original word	Morphological analysis	What to investigate
organization	Derivational noun	Does stem retain lexical identity?
organizer	Derivational noun	Is it merged with organize or organization?
organizing	Inflectional verb form	Is base verb recoverable?
organized	Inflectional form or adjective	Is grammatical context lost?
organizations	Inflectional plural	Is it normalized consistently?
runs	Inflectional verb or plural noun	Is the base form recovered?
running	Inflectional verb or gerund	Is the base verb recovered?
studies	Inflectional plural or verb form	Is spelling change handled?
studied	Inflectional verb form	Is base form recovered?
relational	Derivational adjective	Is intended meaning preserved?
happiness	Derivational noun	Is it confused with happy?
happily	Derivational adverb	Is adverbial meaning lost?
national	Derivational adjective	Does stem remain interpretable?
nationality	Derivational noun	Is meaning distinguishable?
conditional	Derivational adjective	Is useful meaning lost?
condition	Base noun or verb	Does stem fit the word family?
connection	Derivational noun	Is it distinguished from related forms?
connected	Inflectional form or adjective	Is it normalized consistently?
connecting	Inflectional verb form	Is base form recovered?
washable	Derivational adjective	Is property meaning retained?

These are candidate cases for error analysis, not a claim that all 20 are incorrect in every application. Record the generated stem and manually classify the result as acceptable or problematic for the target task.

Conclusion: Tokenizing and stemming each token corrects the programming error. A separate semantic evaluation is needed to determine whether the stems improve or damage retrieval.

Question 6 – Error Analysis of a Finite-State Morphological Parser

The following simplified parser is intended to identify plural nouns.

```
def parser(word):
    if word.endswith("s"):
        return word[:-2], "Plural Noun"
    else:
        return word, "Singular"

words = ["cars","boxes","cities","children"]

for w in words:
    print(parser(w))
```

Tasks

1. Identify the logical errors in the parser.
2. Explain why the parser fails for different plural formation rules.
3. Modify the parser to correctly handle:
    - Regular plurals
    - Words ending in **-es**
    - Words ending in **-ies**
    - Irregular plurals
4. Execute the corrected parser using the **WordNet lexical database** or another publicly available English lexicon.
5. Discuss limitations of rule-based finite-state parsing.

1. Original program and errors

```
def parser(word):
    if word.endswith("s"):
        return word[:-2], "Plural Noun"
    else:
        return word, "Singular"

words = ["cars", "boxes", "cities", "children"]
for w in words:
    print(parser(w))
```

It removes two characters from every word ending in s, although regular plurals usually require removing one.
It does not distinguish the endings -s, -es, and -ies.
It cannot recognize irregular plurals such as children, mice, and people.

2. Corrected parser

```
def parser(word):
    irregular = {
        "children": "child", "men": "man", "women": "woman",
        "mice": "mouse", "geese": "goose", "feet": "foot",
        "teeth": "tooth", "people": "person"
    }
    word = word.lower()

    if word in irregular:
        return irregular[word], "Irregular Plural Noun"

    if word.endswith("ies") and len(word) > 3:
        return word[:-3] + "y", "Plural Noun (-ies)"

    if word.endswith(("ches", "shes", "xes", "zes", "sses")):
        return word[:-2], "Plural Noun (-es)"

    if word.endswith("s") and not word.endswith("ss"):
        return word[:-1], "Plural Noun (-s)"

    return word, "Singular or Unknown"

words = [
    "cars", "boxes", "cities", "children", "books",
    "watches", "babies", "mice", "people", "car"
]
for word in words:
    root, category = parser(word)
    print(f"{word:12} -> {root:10} | {category}")
```

3. Expected output

```
cars         -> car        | Plural Noun (-s)
boxes        -> box        | Plural Noun (-es)
cities       -> city       | Plural Noun (-ies)
children     -> child      | Irregular Plural Noun
books        -> book       | Plural Noun (-s)
watches      -> watch      | Plural Noun (-es)
babies       -> baby       | Plural Noun (-ies)
mice         -> mouse      | Irregular Plural Noun
people       -> person     | Irregular Plural Noun
```

car -> car | Singular or Unknown

4. Using WordNet

```
import nltk
from nltk.corpus import wordnet as wn

nltk.download("wordnet")
nltk.download("omw-1.4")

def wordnet_check(word):
    return len(wn.synsets(word, pos=wn.NOUN)) > 0

for word in ["car", "box", "city", "child", "mouse", "person"]:
    print(word, "->", wordnet_check(word))
```

WordNet can validate candidate lemmas, but it does not automatically make the rule-based parser correct. A lexicon with explicit lemma and inflection relationships or an English morphological analyzer is more suitable for larger-scale work.

5. Limitations

Irregular forms cannot be handled by a finite set of suffix rules alone.
Words ending in s can be plural nouns, verbs, or singular words.
Spelling changes require explicit rules.
New words, technical terms, and names may be missing from the lexicon.
A parser operating on isolated words cannot reliably resolve grammatical roles.
Conclusion: Ordered rules combined with lexical validation improve plural recognition, but rule-based parsing still needs lexical coverage and ambiguity handling.

Question 7 – Error Analysis of Morphological Feature Extraction

The following code is intended to preprocess text before training a machine learning classifier.

```
from nltk.stem import PorterStemmer
from sklearn.feature_extraction.text import CountVectorizer

stemmer = PorterStemmer()

documents = [
    "running runners runs",
    "studies studied studying",
    "organization organized organizer"
]

vectorizer = CountVectorizer()

X = vectorizer.fit_transform(documents)

features = [stemmer.stem(word) for word in vectorizer.get_feature_names_out()]

print(features)
```

Tasks

1. Identify the preprocessing error in the program.
2. Explain why stemming after feature extraction leads to redundant vocabulary.
3. Correct the preprocessing pipeline.
4. Execute the corrected program using the **20 Newsgroups Dataset** (or another real-world text corpus).
5. Compare:
    - Vocabulary size before correction
    - Vocabulary size after correction
    - Classification accuracy
    - Processing time
6. Interpret how the corrected pipeline improves NLP model performance.

1. Identify the preprocessing error

```
from nltk.stem import PorterStemmer
from sklearn.feature_extraction.text import CountVectorizer

stemmer = PorterStemmer()
```

```
documents = [
    "running runners runs",
    "studies studied studying",
    "organization organized organizer"
]
vectorizer = CountVectorizer()
X = vectorizer.fit_transform(documents)
features = [
    stemmer.stem(word)
    for word in vectorizer.get_feature_names_out()
]
print(features)
```

Stemming occurs after CountVectorizer has built the vocabulary and document-term matrix. Stemming the feature names afterward does not combine counts in X, so displayed feature names no longer necessarily correspond to matrix columns.

2. Corrected pipeline and real-world experiment

```
!pip install nltk scikit-learn pandas

import re
import time
import pandas as pd
from nltk.stem import PorterStemmer
from sklearn.datasets import fetch_20newsgroups
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.pipeline import Pipeline
from sklearn.svm import LinearSVC
from sklearn.metrics import accuracy_score

stemmer = PorterStemmer()

def stem_document(text):
    words = re.findall(r"[a-z]+", text.lower())
    return " ".join(stemmer.stem(word) for word in words)

categories = [
    "comp.graphics", "rec.sport.baseball",
    "sci.space", "talk.politics.misc"
]

train_data = fetch_20newsgroups(
    subset="train", categories=categories,
    remove=("headers", "footers", "quotes")
)
test_data = fetch_20newsgroups(
    subset="test", categories=categories,
    remove=("headers", "footers", "quotes")
)

results = []
for use_stemming in [False, True]:
    start = time.perf_counter()

    if use_stemming:
        X_train = [stem_document(t) for t in train_data.data]
        X_test = [stem_document(t) for t in test_data.data]
        strategy = "Stemming before vectorization"
    else:
        X_train = train_data.data
        X_test = test_data.data
        strategy = "No stemming"

    model = Pipeline([
        ("vectorizer", CountVectorizer()),
        ("classifier", LinearSVC())
    ])
```

```
    model.fit(X_train, train_data.target)
    predictions = model.predict(X_test)
    elapsed = time.perf_counter() - start

    vocabulary_size = len(
        model.named_steps["vectorizer"].vocabulary_
    )
    accuracy = accuracy_score(test_data.target, predictions)

    results.append({
        "Strategy": strategy,
        "Vocabulary Size": vocabulary_size,
        "Accuracy": accuracy,
        "Processing Time (seconds)": elapsed
    })

print(pd.DataFrame(results).to_string(index=False))
```

The first dataset download requires internet access in the execution environment. If unavailable, download the corpus separately or use a local corpus.

3. Demonstrate vocabulary reduction

```
from sklearn.feature_extraction.text import CountVectorizer
from nltk.stem import PorterStemmer
import re

documents = [
    "running runners runs",
    "studies studied studying",
    "organization organized organizer"
]
stemmer = PorterStemmer()

def stem_document(text):
    words = re.findall(r"[a-z]+", text.lower())
    return " ".join(stemmer.stem(word) for word in words)

original_vectorizer = CountVectorizer()
X_original = original_vectorizer.fit_transform(documents)

stemmed_documents = [stem_document(doc) for doc in documents]
stemmed_vectorizer = CountVectorizer()
X_stemmed = stemmed_vectorizer.fit_transform(stemmed_documents)

print("Vocabulary before stemming:")
print(sorted(original_vectorizer.get_feature_names_out()))
print("\nVocabulary after stemming:")
print(sorted(stemmed_vectorizer.get_feature_names_out()))
print("\nVocabulary size before:", X_original.shape[1])
print("Vocabulary size after:", X_stemmed.shape[1])
```

4. Performance comparison

Metric	No stemming	Stemming before vectorization
Vocabulary size	Measure from fitted vectorizer	Measure from fitted vectorizer
Classification accuracy	Measure on test set	Measure on same test set
Processing time	Measure using timer	Includes stemming and vectorization
Semantic information	Retains original forms	Some word-form distinctions are lost

No numerical values are provided here because they depend on the downloaded corpus, software environment, and execution time.

5. Interpretation

Vocabulary size may be reduced.
Fewer sparse, redundant features may improve statistical support for related word forms.
Generalization may improve when training data is limited.
Stemming may also reduce accuracy if semantically different words are merged.
Conclusion: Stemming should be performed before vectorization if the goal is to combine word-form counts. Choose a pipeline based on test-set accuracy, vocabulary size, processing time, and the semantic information required by the task.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Error Identification	20%	Accurately identifies all errors with clear understanding of issues	Identifies most errors with minor omissions	Identifies limited errors; some are missed	Fails to identify key errors
Root Cause Analysis	30%	Clearly explains underlying causes with strong technical reasoning	Explains causes with moderate clarity	Partial explanation; weak reasoning	Unable to identify causes of errors
Correction & Justification	25%	Provides correct solutions with strong justification and logic	Provides correct corrections with some justification	Corrections are partially correct or weakly justified	Incorrect or no corrections provided
Application of Concepts	15%	Effectively applies relevant concepts to analyze and resolve errors	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts
Clarity & Presentation	10%	Presents analysis clearly, logically, and systematically	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand

Q. No. / Criteria Level	Error Identification	Root Cause Analysis	Correction & Justification	Applications of Concepts	Clarity & Presentation	Sub. Total	Total %
1							
2							
3							
4							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____